

Design: closing the §6.AK Check 10 self-reference gap

Date: 2026-09-22 **Status:** Draft — design only, NOT implemented
Operator directive (implicit): resolve the "choice belongs to the operator" question docs/scripts/check-cycle-coverage.sh.md §5 has left open since LVA-149 (2026-08-26). **Forensic anchor:** .lava-ci-evidence/sixth-law-incidents/2026-09-22-cycle-coverage-self-reference-push-bypass.json — the session that required `git push --no-verify` to ship two already-Gate-7-passed, real Firebase distributes (client 1.3.17-1087, api-app 0.2.13-28) past this exact gap.

Classification: universal (the self-reference problem is inherent to any content-addressed-commit gate that binds evidence to "the commit currently being pushed"; the specific file paths are Lava's).

1. The problem, restated precisely

scripts/check-cycle-coverage.sh requires the §6.Z evidence file to declare a commit SHA that is a **prefix match** of `--head` (see the script's `sha_bound` loop, lines ~152–161). Two callers invoke it:

- scripts/firebase-distribute.sh Gate 7 passes `--head` = the commit that already carries both the code and the evidence — this binds cleanly by construction, and it is the load-bearing check: it actually gates the real Firebase upload, and it genuinely ran and PASSED twice this session.
- .github/pre-push Check 10 passes `--head` = `git rev-parse HEAD` at push time, which is the **pointer-advancing commit** (the commit that moves `last-version-debug/last-version-release`). A commit cannot embed its own SHA — that would require a git object to contain a hash of itself, which content-addressing makes impossible by construction, not by a parsing bug.

This is **not the LVA-149 defect** (that was a parser that silently accepted "unknown" as a pass). This is a structural mismatch between what two different call sites re-check.

2. Why the underlying safety property is not actually violated here

§6.J's anti-bluff floor is "don't let a distribute ship without genuinely-executed, passing device-Challenge evidence for the exact code being shipped" (CLAUDE.md §6.J, §6.Z clause 2, §6.AK clause 1). That property was upheld this session: Gate 7 ran against the real commit, found real PASS records, and only THEN did `firebase-distribute.sh` upload the real APK to Firebase. Check 10's refusal is not catching a bluff — it is re-deriving the same true fact from a commit that structurally cannot state it, and failing to re-derive it for a reason that has nothing to do with whether the artifact is safe.

A fix must preserve exactly this: **any DIFFERENCE in production code between the evidence's declared commit and the pointer-advance commit MUST still refuse**. Only the identical-code case may pass.

3. Recommended option: 1, implemented via 2's mechanism (not 3)

Pick a hybrid of options 1 and 2 from `docs/scripts/check-cycle-coverage.sh.md` §5: Check 10 should conceptually "gate the code commit, not the pointer-advance commit" (option 1's framing), and the mechanism that makes this safe without inventing a new insecure code path is option 2's ancestor-plus-diff-scope condition.

Concretely: teach `check-cycle-coverage.sh` a **new resolution step**, used only when the literal-match check at `--head` fails:

1. Read the evidence file's declared SHA(s) as today (`ESHAS[]`).
2. For each declared SHA `S` that is a **valid, resolvable ancestor of --head** (`git merge-base --is-ancestor "$S" "$HEAD"`), compute `git diff --name-only "$S" "$HEAD"`.
3. If every changed path matches `^(\\.lava-ci-evidence/|docs/|CHANGELOG\\.md)` (a fixed, narrow allowlist — no production source, no build config, no scripts), treat the binding as held: `sha_bound=1`, but record which SHA satisfied it and log it distinctly from a literal match (so §6.AK PASS output is honest about which path fired).

4. If `S` is not an ancestor, or the diff touches anything outside the allowlist, the check falls through to today's behavior: refuse with §6.AK.4, unchanged.

This is deliberately **not** a general "ancestor is good enough" rule — an ancestor whose tree differs in `app/build.gradle.kts` or any proguard file must still refuse, because that would be exactly the class of bluff §6.AK exists to prevent (evidence binding to a DIFFERENT build than the one shipping). The allowlist is what makes this "code-identity, not leniency," per option 2's own framing.

3.1 Why not straight Option 1 (gate literally the code commit)

Doing this would require `.githubhooks/pre-push` to know, out of band, which of `N` commits about to be pushed is "the code commit" versus "the pointer-advance commit" — there is no existing signal for that distinction other than re-deriving it from the evidence file itself, which is exactly what the option-2 mechanism above does. A bare "Option 1" that changes `--head` to some other git ref (e.g. `HEAD~1`) is fragile: it silently assumes a fixed commit-count offset between the code commit and the push, which breaks the moment a session does what this one did — split code and evidence into two commits, then need a third commit (docs-sync fix) between them. The mechanism above is offset-independent: it walks ancestry, not commit count.

3.2 Why not Option 3 (leave it strict, evidence-at-distribute-time-only, forever)

This is the CURRENT de facto state, and it is the reason this session needed `--no-verify`. It does not scale: §11.4.18/CM-SCRIPT-DOCS-SYNC (a completely unrelated, legitimate gate) can force a docs-only follow-up commit after the evidence commit, and any such follow-up permanently and correctly-by-design breaks Check 10 for that push, forcing `--no-verify` every single time a housekeeping commit lands between the evidence commit and push. Per §6.J, routine reliance on `--no-verify` is itself the failure mode ("Routine bypass (`--no-verify`) is itself a Seventh Law incident" — CLAUDE.md, Local-Only CI/CD section). Option 3 trades a real safety gap for a permanent, expected, "normal" bypass — worse than fixing it.

4. Concrete implementation sketch

File to change: scripts/check-cycle-coverage.sh, the block at lines ~152–161 (the sha_bound loop).

```
sha_bound=0
bound_via=""
for s in "${ESHAS[@]"}; do
    if [[ "$HEAD" == "$s"* || "$s" == "$HEAD"* ]]; then
        sha_bound=1; bound_via="literal:$s"; break
    fi
done

# NEW: ancestor-plus-code-identity fallback (see design doc
#       2026-09-22).
# Only tried when no literal match exists – this NEVER weakens the
#       literal
# path, it only widens it for the pointer-advance self-reference
#       case.
if [[ "$sha_bound" -ne 1 ]]; then
    for s in "${ESHAS[@]"}; do
        # Full 40-char SHA required for merge-base/diff – reject
        # short SHAs here
        # (the literal-match path above already accepts short
        # prefixes; this
        # fallback needs the resolvable object).
        git -C "$REPO_ROOT" cat-file -e "$s" 2>/dev/null ||
            continue
        git -C "$REPO_ROOT" merge-base --is-ancestor "$s" "$HEAD"
        2>/dev/null || continue
        # Every changed path between the declared ancestor and
        # HEAD must be
        # governance/evidence-only – any production file refuses.
        if ! git -C "$REPO_ROOT" diff --name-only "$s" "$HEAD" 2>/
            dev/null \
                | grep -vE '^(\.lava-ci-evidence/|docs/|
            CHANGELOG\.md$)' -q; then
            sha_bound=1; bound_via="ancestor-code-identity:$s";
            break
        fi
    done
fi

if [[ "$sha_bound" -ne 1 ]]; then
    echo "FATAL $6.AK.4: $6.Z evidence commit-SHA does not match
    the commit under test." >&2
    echo "    evidence '$EVI' declares: ${ESHAS[*]}" >&2
    echo "    current HEAD:                $HEAD" >&2
    echo "    (checked: literal prefix match, and ancestor-
    with-governance-only-diff – neither held)" >&2
    exit 2
fi
```

And in the final PASS line, surface `$bound_via` so an auditor can see which path fired:

```
echo "$6.AK PASS: $claims claim(s) / $refs_total covering ref(s)
        verified against $REC_COUNT parsed verdict record(s) in
        $EVI (SHA $HEAD, bound via $bound_via)"
```

5. New hermetic test cases (template: `tests/cycle-coverage/test_cycle_coverage.sh`)

That file already builds synthetic fixtures via a `write_evidence()` helper (see its positive/skip/stale/host-direct cases at lines ~100–133) and drives the script with `LAVA_CYCLE_COVERAGE_HEAD/- -now-epoch` pinned for determinism. Add a new case group, `EDIR_ANCESTOR_*`, following the exact same shape:

1. **`test_ancestor_governance_only_diff_passes`** — build two real commits in a throwaway git repo fixture (this file already needs *some* git state for merge-base/diff to work, unlike today's fully-flag-pinned tests — check whether the existing harness already has a git fixture elsewhere in `tests/`, e.g. `tests/check-constitution/'s make_fixture()` pattern, and reuse that convention rather than inventing a second one). Commit A changes only `.lava-ci-evidence/x + docs/y.md`; write evidence in commit A declaring `commit_sha = commit A's real SHA`. Set `--head = commit A's own hash` for one assertion (literal match, unchanged behavior) AND `--head = a synthetic "commit B"` that is a pure-docs child of A for the new behavior. Expect exit 0, and expect the PASS line to contain `bound via ancestor-code-identity`.
2. **`test_ancestor_with_production_diff_refuses`** — identical setup, but the child commit B ALSO touches `app/build.gradle.kts` (or any non-allowlisted path). Expect exit 2, `$6.AK.4`, and confirm the ancestor path was tried and rejected (not silently skipped).
3. **`test_ancestor_non_ancestor_sha_still_refuses`** — evidence declares a real, resolvable SHA that is NOT an ancestor of `--head` (e.g. a sibling branch tip). Expect exit 2 — proves the fallback doesn't accidentally accept any resolvable SHA, only true ancestors.
4. **Regression: all 6 existing negative cases + the 1 positive case in the current suite must still pass unchanged** — the new fallback must be provably inert whenever a literal match already exists (this is why `bound_via` is logged: the existing positive-case assertion should be tightened to also assert `bound via literal:`, proving the new code path did not silently become the ONLY path that fires).

6. What this design doc does NOT do

Nothing here is implemented. `scripts/check-cycle-coverage.sh`, `.githubhooks/pre-push`, and `tests/cycle-coverage/test_cycle_coverage.sh` are all unmodified by this doc. The standing, current state remains:

- `firebase-distribute.sh` Gate 7 is the load-bearing check and requires no change — it already binds correctly by construction.
- `.githubhooks/pre-push` Check 10 has the self-reference gap described above. Until section 4's sketch is implemented and proven via section 5's test cases, **any future distribute cycle whose evidence-authoring commit is not the literal push-time HEAD will require an explicit, operator-authorized `git push --no-verify`**, with an incident record citing this design doc and `.lava-ci-evidence/sixth-law-incidents/2026-09-22-cycle-coverage-self-reference-push-bypass.json`, exactly as this session did.
- A future session implementing this MUST update `docs/scripts/check-cycle-coverage.sh.md` §5 in the same commit (CM-SCRIPT-DOCS-SYNC) to remove the "open design question" framing and document the resolved behavior, including the `bound_via` field's two possible values.